PSYC 51.17: Models of Language and Communication

Lecture 17: Training Transformers

Week 5, Lecture 3 - From Architecture to Implementation

PSYC 51.17: Models of Language and Communication

Winter 2026

Winter 2026

Today's Agenda

1. **Positional Encoding**: Injecting sequence order
2. **Feed-Forward Networks**: The other key component
3. **Layer Norm & Residuals**: Training deep networks
4. **FlashAttention**: Making transformers faster
5. **Three Architectures**: Encoder, Decoder, Both
6. **Practical Implementation**: Using HuggingFace

Goal: Complete understanding of transformer training and implementation

Winter 2026

Positional Encoding

Problem: Self-attention is permutation-invariant!

Without position information:

- "The cat sat" = "sat cat the" = "cat the sat"
- Order matters in language!

Solution: Add positional encodings to input embeddings

Original Transformer

Concrete Example:

Why Sinusoidal Positional Encoding?

Advantages of sine/cosine functions:

1. Unique Patterns

- Each position gets unique vector
- Different frequencies per dimension

Visualization:

```
Dim 0 (high freq): ~~~~~ (fast oscillation)
Dim 1 (mid freq): ~~~
```

PSYC 51.17: Models of Language and Communication

Why Sinusoidal Positional Encoding?

```
11 | Each position has a unique "barcode"!
```

Winter 2026

...continued

Visualizing Positional Encodings

Each position gets a unique pattern across dimensions

```
Pos 0 → Pos 5 → Pos 9 → Dim 0 → Dim 4 → Dim 7
```

Key Insight:

- Lower dimensions: Rapid oscillation (high frequency)
- Higher dimensions: Slow oscillation (low frequency)
- Creates a unique "barcode" for each position
- Model learns to use these patterns for positional awareness

Feed-Forward Networks

After attention, apply position-wise feed-forward network

Architecture: Two linear layers with ReLU/GELU activation

```
class FeedForward(nn.Module):
```

Purpose: Add non-linearity and increase model capacity

- Attention is mostly linear (weighted sums)
- FFN adds expressiveness through ReLU/GELU

2. **Position-wise Processing**

- Attention mixes information across positions
- FFN processes each position independently
- Allows position-specific feature transformations

3. **Increase Model Capacity**

- Expansion (4x) provides more parameters
- Can learn complex feature combinations

Layer Normalization & Residual Connections

Critical for training deep transformers!

Residual Connections:

```
# output = sublayer(x) + x
x = x + self.attention(x)
x = x + self.feedforward(x
```

Layer Normalization:

```
# Normalize across features (not batch)
def layer_norm(x
```

Pre-Norm vs Post-Norm

Two ways to arrange LayerNorm and residual connections

Post-Norm (Original):

```
# Normalize AFTER residual
x = LayerNorm(x + Attention(x))
x = LayerNorm(x + FFN(x))
```

- Used in original Transformer, BERT
- Can be unstable for deep models
- Requires careful initialization

Pre-Norm (Modern):

```
# Normalize BEFORE sublayer
x = x + Attention(LayerNorm(x))
x = x + FFN(LayerNorm(x))
```

- Used in GPT-2, GPT-3, LLaMA
- More stable gradients
- Easier to train 96+ layer models

Complete Transformer Block

Putting it all together:

```
class TransformerBlock(nn.Module):
```

Model sizes:

- BERT-base: 12 blocks, 110M params | BERT-large: 24 blocks, 340M params
- GPT-3: 96 blocks, 175B params!

FlashAttention: Making Transformers Faster

Problem: Standard attention is slow and memory-hungry!

Standard Attention Complexity

- Time: O(n^2) where n = sequence length
- Memory: O(n^2) to store attention matrix
- Bottleneck: Reading/writing to GPU memory (HBM)

FlashAttention Innovation:

- Tile-based computation
- Uses fast SRAM instead of slow HBM
- Fused operations (fewer memory reads)
- Recomputation in backward pass

Concrete Speedup:

Method	Speed	Memory
Standard	1x	1x
FlashAttention	3x faster	0.5x

Real Impact:

- Sequence 1024→4096 on same GPU

- Approximate attention with linear complexity
- Kernel trick to avoid materializing attention matrix
- Used in: Performer, Linear Transformer

3. **Low-Rank Approximation**

- Factorize attention matrix
- Reduce memory footprint

Three Transformer Architectures

p{4cm}p{4cm}} Architecture	How it works	**Examples \	Uses**
Best for: Classification, NER, QA			
Best for:			

Visual Comparison: Encoder vs Decoder vs Both

```
**Encoder-Only (BERT) → Self-Attention → bidirectional →
```

\end{center**

Choosing the Right Architecture:

- Need to understand full context? → **Encoder** (BERT)
- Need to generate text? → **Decoder** (GPT)
- Need both (translate, summarize)? → **Encoder-Decoder** (T5)

Using Transformers in Practice

HuggingFace makes it easy!

```python
from transformers import AutoModel, AutoTokenizer

# Load pre-trained model (downloads ~440MB first time)
model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

# Tokenize input
text = "The animal didn't cross the street because it was too tired"
inputs = tokenizer(text, return_tensors="pt")
print(inputs['input_ids'])
# tensor([[ 101, 1996, 4111, 2134, 1005, 1056, 2892, 1996,
# 2395, 2138, 2009, 2001, 2205, 5458, 102]])
# [CLS] The animal didn ' t cross the ...

# Get contextualized embeddings
```


continued...

Using Transformers in Practice

```
outputs = model(**inputs)
hidden_states = outputs.last_hidden_state # [1, 15, 768]

# "it" is at position 10 - its embedding knows it refers to "animal"!
it_embedding = hidden_states[0, 10, :] # 768-dim context-aware vector
```

Reference: HuggingFace Course - Chapter 1.4

Winter 2026

...continued

Comparing Architectures: Code Examples

Different architectures for different tasks

```
from transformers import AutoModelForSequenceClassification, \
 AutoModelForCausalLM, \
 AutoModelForSeq2SeqLM

# 1. ENCODER-ONLY (BERT): Classification
encoder_model = AutoModelForSequenceClassification.from_pretrained(
 "bert-base-uncased", num_labels=2
)
# Use for: Sentiment analysis, NER, classification

# 2. DECODER-ONLY (GPT): Text generation
decoder_model = AutoModelForCausalLM.from_pretrained("gpt2")
# Use for: Text completion, creative writing, few-shot learning

# 3. ENCODER-DECODER (T5): Seq2Seq tasks
seq2seq_model = AutoModelForSeq2SeqLM.from_pretrained("t5-base")
# Use for: Translation, summarization, question answering
```


continued...

Comparing Architectures: Code Examples

```
18
19  # All use transformer architecture, different attention patterns!
```

Key Takeaway: Choose architecture based on your task!

Winter 2026

...continued

Practical Tips for Training Transformers

1. Learning Rate & Warmup

```
# Warmup: gradually increase LR
# Then decay (linear or cosine)
scheduler =
get_linear_schedule_with_warmup
 optimizer,
 num_warmup_steps=1000, # ~10%
of training
 num_training_steps=10000
)
# Fine-tuning: lr=2e-5, From
scratch: lr=1e-4
```

3. Regularization

```
# Dropout after attention and
FFN
self.dropout = nn.Dropout(0.1)

# Gradient clipping
torch.nn.utils.clip_grad_norm_(
 model.parameters(),
max_norm=1.0
)
```


4. Mixed Precision (FP16)

- Shorter sequences train faster (quadratic complexity!)
- Consider truncation or sliding windows
- Pack multiple examples to maximize GPU usage

3. **Model Size**

- Start small, scale up if needed
- DistilBERT: 40% smaller, 60% faster, 97% performance

2. **Architecture Choice:**

- When would you use encoder-only vs decoder-only?
- Can GPT do classification? Can BERT generate?
- Why has decoder-only become more popular recently?

3. **Scaling:**

- Is bigger always better?
- What are the limits to scaling transformers?

Next week (Week 8):

- Masked Language Modeling
- Pre-training and fine-tuning
- Contextual embeddings in action

\item

- RoBERTa, ALBERT, DistilBERT
- Improvements and optimizations

\item

- Position-wise transformations
- Add non-linearity and capacity

3. **LayerNorm & Residuals**

- Critical for training deep networks
- Stabilize gradients, enable deeper models

4. **Three Architectures**

\item **Su et al. (2021)** - "RoFormer: Enhanced Transformer with Rotary Position Embedding"

- Modern positional encoding approach

\item **Dao et al. (2022)** - "FlashAttention: Fast and Memory-Efficient Exact Attention"

- Making transformers faster

Tutorials:

Questions?

Discussion Time

Topics for discussion:

- Positional encoding approaches
- Architecture choices
- Training tips and tricks
- Implementation questions
- Assignment 4 preparation

Thank you!

See you next week for BERT!